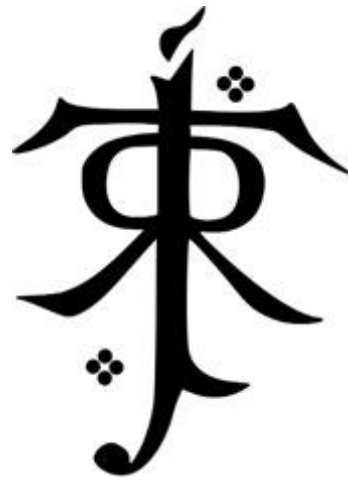




Token Efficient Language

Diseño e Implementación de un Lenguaje

Equipo

Nombres	Apellido	Legajo	E-mail
Bautista	Pessagno	65101	bpessagno@itba.edu.ar
Rodrigo Alejandro	Hernandez	65522	rohernandez@itba.edu.ar
Lorenzo Alejandro	Méndez	65147	lmendez@itba.edu.ar
Pascal	Ordano	65485	pordano@itba.edu.ar

Tabla de contenidos

1. Introducción.....	3
2. Modelo Computacional.....	4
2.1 Dominio.....	4
Medición del ahorro de tokens.....	4
2.2. Lenguaje.....	6
Tipos y declaraciones.....	6
Funciones y retornos.....	6
Bloques por indentación y control de flujo.....	7
Punteros, arreglos y punteros a función.....	7
Literales, comentarios e inline C.....	8
Directivas y punto de entrada.....	8
3. Implementación.....	8
3.1. Frontend.....	9
Análisis léxico (Flex).....	9
Análisis sintáctico (Bison).....	10
Árbol de Sintaxis Abstracta.....	10
3.2 Backend.....	10
Análisis semántico.....	11
Generación de código C.....	12
3.3 Adicionales.....	13
Pruebas.....	13
Programas de demostración.....	14
3.4 Dificultades Encontradas.....	14
4. Limitaciones.....	15
5. Futuras Extensiones.....	16
6. Conclusiones.....	16
7. Referencias.....	17
8. Bibliografía.....	17

1. Introducción

TEL (*Token Efficient Language*) es un lenguaje de dominio específico, de sintaxis similar a Python, que compila a lenguaje C. Su objetivo es representar programas escritos en un subconjunto de C con una sintaxis más compacta, de modo de minimizar la cantidad de *tokens* consumidos por los Modelos de Lenguaje de Gran Escala (LLMs), sin perder información semántica y manteniendo la legibilidad para un desarrollador.

La motivación es concreta. Cuando un desarrollador utiliza un agente (Claude, GPT, Gemini, entre otros) para escribir, revisar o modificar código C, la verbosidad inherente del lenguaje C genera un consumo excesivo de tokens, tanto en la entrada (los *prompts*, la lectura de archivos y la salida de comandos) como en la salida (las respuestas del modelo). Ese consumo impacta de forma directa en tres dimensiones: el costo económico, ya que los proveedores cobran por token; la latencia, dado que más tokens implican más tiempo de respuesta; y la ventana de contexto disponible, porque cada token gastado en sintaxis es un token menos para contenido semántico.

El Stage III entrega un compilador funcional que recorre el pipeline completo: lee un programa TEL desde la entrada estándar, lo lexa y parsea con Flex y Bison, construye un Árbol de Sintaxis Abstracta (AST), valida su semántica y, finalmente, emite código C estándar, determinista y compilable por la salida estándar. Se trata, por lo tanto, de un compilador fuente a fuente (*source-to-source*).

Este documento describe el desarrollo y la concepción de las ideas detrás del lenguaje y de su compilador.

2. Modelo Computacional

2.1 Dominio

El dominio de TEL es la optimización de tokens en la interacción humano-LLM para el desarrollo de software en C. El compilador toma como entrada un programa en TEL y produce como artefacto de salida su equivalente en código C estándar.

La transformación se concibe como biyectiva en su definición conceptual:

dado cualquier programa válido en TEL, el compilador produce un único programa C equivalente y bien formado y, recíprocamente, cualquier programa dentro del subconjunto de C soportado tiene una representación única en TEL. Para el alcance de este proyecto se implementa únicamente la dirección $TEL \rightarrow C$.

El subconjunto de C cubierto por TEL incluye:

1. tipos primitivos (`int`, `char`, `float`, `double`, `void`, `long`, `short`) y los tipos compuestos compactados `uint` y `uli`;
2. calificadores de tipo (`unsigned`, `long`, `short`, `const`);
3. variables y constantes;
4. arreglos y punteros, incluidos los punteros a función;
5. funciones, prototipos y punto de entrada (`main`);
6. estructuras de control (`if/elif/else`, `for`, `while`, `do-while`, `switch`);
7. estructuras agregadas (`struct`, `union`), enumeraciones (`enum`) y alias de tipo (`typedef`);
8. directivas de preprocesador básicas (`#include`, `#define`).

Medición del ahorro de tokens

El valor del lenguaje se evidencia al comparar el conteo de tokens de un programa en C contra su equivalente en TEL. Las siguientes mediciones se obtuvieron con el tokenizador de OpenAI (platform.openai.com/tokenizer), sobre el cuerpo del programa, es decir, sin contar las instrucciones previas que enseñan TEL al modelo.

1. *Hola mundo* con `main`: 27 $\rightarrow$ 8 tokens (99 $\rightarrow$ 29 caracteres).
2. Función que recorre un arreglo con un bucle `for`: 45 $\rightarrow$ 32 tokens (142 $\rightarrow$ 103 caracteres).
3. Definición de un `struct` con `typedef`: 26 $\rightarrow$ 21 tokens (126 $\rightarrow$ 84 caracteres).
4. Cinco directivas `#include`: 26 $\rightarrow$ 16 tokens (101 $\rightarrow$ 80 caracteres).

Sobre la metodología de medición conviene precisar varios puntos:

1. **Un token no equivale a una palabra.** Un LLM determina la *tokenización* según cómo el modelo la haya interpretado durante su entrenamiento; una misma palabra puede ser un token o varios. Por eso las mediciones se hacen con un tokenizador real y no contando palabras.
2. **El costo de los metadatos debe separarse del payload medido.** Para que el LLM entienda la representación compacta, normalmente se le entregan instrucciones

previas (qué significa cada keyword de TEL). Si esas instrucciones se contaran dentro de cada medición, el ahorro aparente sería menor. La medición correcta separa el costo de instrucciones y metadatos del payload TEL en sí. Esto es análogo al problema de la compresión: comprimir achica el payload, pero se requieren metadatos para decodificarlo.

Como complemento a los ejemplos puntuales de la Tabla 1, se midió el ahorro de tokens sobre un conjunto de ocho programas más extensos (resoluciones de problemas de estilo LeetCode, disponibles en demo/leetcode/ y descritos en la sección 3.3) comparando el código TEL (sin comentarios) contra el C generado por el compilador (sin comentarios), con el mismo tokenizador ([cl100k_base](https://platform.openai.com/tokenizer), compatible con platform.openai.com/tokenizer).

Programa	Problema	Tokens TEL	Tokens C	Ahorro	%
two-sum.tel	LC 1 – Two Sum	121	186	65	34.9%
fizzbuzz.tel	LC 412 – Fizz Buzz	93	145	52	35.9%
fibonacci.tel	LC 509 – Fibonacci	96	147	51	34.7%
binary-search.tel	LC 704 – Binary Search	170	223	53	23.8%
reverse-integer.tel	LC 7 – Reverse Integer	114	147	33	22.4%
palindrome-number.tel	LC 9 – Palindrome Number	118	151	33	21.9%
max-subarray.tel	LC 53 – Maximum Subarray (Kadane)	194	262	68	26.0%
climbing-stairs.tel	LC 70 – Climbing Stairs	100	153	53	34.6%
Total		1006	1414	408	28.9%

Tabla 1 - Mediciones del uso de tokens

2.2. Lenguaje

Esta sección describe la superficie del lenguaje.

Tipos y declaraciones

Los tipos de C se reemplazan por variantes que ocupan un único token. Las declaraciones invierten el orden respecto de C, anteponiendo el nombre al tipo mediante `nombre:tipo`.

```
int -> int      char -> char
float -> float   double -> double
long -> long     short -> short
void -> void
uint -> unsigned int  uli -> unsigned long int
```

```
x:int    // int x;
y:uli = 5 // unsigned long int y = 5;
```

La compactación de tipos compuestos es la que más ahorra: `unsigned long int` son tres tokens en C y `uli` es uno solo en TEL.

Funciones y retornos

Las funciones se declaran con `fn nombre args -> tipo_retorno`, donde `args` es una lista de pares `nombre:tipo` separados por espacios, y la flecha con el tipo de retorno puede omitirse cuando la función retorna `void`. El retorno se expresa con `ret`. Si la última sentencia del cuerpo es un identificador suelto o una expresión de comparación o lógica, se la trata como un `ret` implícito.

```
fn foo arg1:int arg2:char -> int // int foo(int arg1, char arg2);
fn bar x:int                // void bar(int x);
```

```
fn max a:int b:int -> int
  a > b           // equivale a: ret a > b
```

Bloques por indentación y control de flujo

En lugar de llaves {}, los bloques se delimitan por indentación, al estilo de Python. El control de flujo usa keywords cortas y, en algunos casos, asume valores por omisión frecuentes en la programación.

```
if -> if      elif -> else if    else -> else
for i 0 n     // for (int i = 0; i < n; i++)
for i = 0, c, i++ // for (int i = 0; c; i++)
while -> while  dw -> do while
break -> break  cnt -> continue
```

El `switch` admite dos variantes, distinguidas por el separador de caso. Con `:` los casos caen al siguiente (*fall-through*) salvo `break` explícito; con `->` cada caso termina implícitamente, sin caída.

switch variable	switch variable
1: do_one()	1 -> do_one()
2: do_two()	2 -> do_two()
break	3 -> do_three()
3: do_three()	default -> do_default()
default: do_default()	

Punteros, arreglos y punteros a función

Los punteros usan `*` y los arreglos usan `[]`, con el nombre antepuesto al tipo. Los literales de arreglo separan sus elementos con espacios, no con comas. Los punteros a función usan la sintaxis `fn*`.

```
arr:int[10]      // int arr[10];
arr:int[3] = {1 2 3} // int arr[3] = {1, 2, 3};
fn* int -> int    // puntero a función (int) -> int
fn* int int -> void // puntero a función (int, int) -> void
fn*              // puntero a función () -> void
```

Literales, comentarios e inline C

Los literales son los de C, con una diferencia: los octales usan el prefijo `0o` (estilo Python) en vez del `0` desnudo de C. Se admiten comentarios de línea `//` y de bloque `/* */`. El literal `null` se traduce al puntero nulo.

```
42  0xFF  0o77  3.14  1.0e-3
'a'  '\n'  "hello" null
```

Se permite embeber código C dentro de bloques delimitados por comillas invertidas (```). El inline C se copia como bloque opaco y no es verificado por TEL.

Directivas y punto de entrada

`#define` se mantiene igual. En `#include`, los headers del sistema se escriben sin extensión ni corchetes: `#include stdio` produce `#include <stdio.h>`. A lo sumo puede aparecer un `main` por unidad de traducción; cuando está presente, se traduce a `int main(int argc, char * argv[])`. Una unidad de traducción puede omitir `main` para compilarse como biblioteca u objeto.

3. Implementación

El compilador está escrito en C y construido con Flex (analizador léxico) y Bison (analizador sintáctico). El pipeline ejecuta cinco fases ordenadas:

```
stdin -> análisis léxico -> análisis sintáctico (AST)
      -> análisis semántico -> generación de código C -> limpieza
```

Si fallan el análisis léxico, el sintáctico o el semántico, el programa termina con estado distinto de cero y no emite un artefacto C usable. Si todas las fases pasan, emite C determinista y sintácticamente válido (salvo en los casos mencionados en las limitaciones). La orquestación vive en `EntryPoint.c`, que inicializa cada módulo (cada `initializeXModule()` devuelve un destructor que se ejecuta en orden inverso), corre `executeSyntacticAnalysis()` y, si tiene éxito, `executeSemanticAnalysis()` y `executeCodeGeneration()`. El estado se transporta en `CompilerState`, una estructura mínima que sostiene la raíz del AST.

3.1. Frontend

El frontend convierte el texto de entrada en un AST.

Análisis léxico (Flex)

El lexer (`FlexPatterns.l` y `FlexActions.c`) reconoce el alfabeto completo de TEL: literales, las keywords del lenguaje, los nueve tipos primitivos, el conjunto de operadores aritméticos, de asignación, bit a bit, lógicos y de comparación, los delimitadores, y las directivas `#include` y `#define`.

El lexer utiliza varias *start conditions* de Flex para resolver ambigüedades:

1. **BOL** (*beginning of line*) procesa la indentación al inicio de cada línea y genera los tokens `INDENT` y `DEDENT`.
2. **AFTER_POSTFIX** se activa luego de un identificador o de un cierre (`()` o `]`) para interpretar correctamente el operador `->` y distinguirlo de la flecha de tipo de retorno de las funciones.
3. **INLINE_C** y **BLOCK_COMMENT** manejan, respectivamente, los bloques de código C embebido (delimitados por comillas invertidas) y los comentarios de bloque `/* */`.

La indentación significativa se implementa con una pila de niveles (`StackADT.c`, una pila de enteros con capacidad inicial 128 y base 0). Un tabulador cuenta como cuatro espacios y la indentación debe ser múltiplo de cuatro; en caso contrario el análisis falla. Cuando el nivel sube, se apila el nuevo nivel y se emite `INDENT`; cuando baja, se desapila hasta encontrar un nivel coincidente, emitiendo un `DEDENT` por cada salto. Al llegar al fin de archivo se emite un terminador, se vacían los `DEDENT` pendientes hasta la base y se emite el token de fin. El lexer también reetiqueta como `TYPEDEF_NAME` a los identificadores que el parser ya registró como nombres de `typedef`, lo que desambigua su uso como tipo.

Análisis sintáctico (Bison)

La gramática (`BisonGrammar.y`) está libre de contexto y usa un *push parser*. Cada token producido por Flex empuja un paso de parseo mediante `yypush_parse()`, integrado en el bucle de `Frontend.c`. La gramática define precedencia y asociatividad al estilo de C, de la asignación (la más baja, asociativa a derecha) hasta los operadores unarios y de postfijo (la más alta). Los errores de sintaxis se informan con mensajes detallados.

La estructura de la gramática parte de `program`, formado por items de nivel superior (declaraciones de variables y funciones, `main`, agregados, enums, typedefs, directivas e inline C) y, dentro de las funciones, por sentencias (declaraciones, `ret`, expresiones, control de flujo, `break/cnt`).

Árbol de Sintaxis Abstracta

Las acciones semánticas (`BisonActions.c`) asignan cada nodo con `calloc` y encadenan las listas (parámetros, sentencias, expresiones, campos) como listas enlazadas. El AST (`AbstractSyntaxTree.h`) clasifica sus nodos mediante varias enumeraciones: `TypeKind` (primitivos más `NAMED`, `STRUCT`, `ENUM`, `UNION`, `POINTER`, `ARRAY`, `FUNCTION_POINTER`), `ProgramItemKind`, `StatementKind`, `ExpressionKind` y `ExpressionOperator`. Cada acción de declaración de `typedef` registra su nombre, lo que habilita el reetiquetado posterior en el lexer.

Una particularidad es el retorno implícito: al cerrar el cuerpo de una función, si la última sentencia es una expresión que consiste en un identificador o en operadores de comparación o lógicos, se la convierte en una sentencia de retorno. Los `DEDENT` de cierre y el manejo del cuerpo viven en estas acciones.

3.2 Backend

El backend opera sobre el AST en dos fases: análisis semántico y generación de código.

Análisis semántico

El analizador (`SemanticAnalyzer.c`, apoyado en `SemanticSymbolTable.c`) recorre el AST con un contexto, `SemanticAnalysisContext`, que sostiene la tabla de símbolos, el tipo de retorno de la función en curso, la profundidad de bucles y de `switch` (para validar `break` y `cnt`) y una bandera de error acumulado. La validación se realiza en dos pasadas:

1. **Colección global.** Registra todos los símbolos de nivel superior (variables globales, funciones y sus prototipos, agregados, enums, typedefs y constantes de enum) sin validar todavía los cuerpos. Las redeclaraciones de funciones deben tener firmas compatibles y los duplicados se rechazan.

2. **Validación.** Recorre los cuerpos de las funciones y las sentencias, validando tipos, expresiones, llamadas, retornos y control de flujo.

La tabla de símbolos implementa *scopes* léxicos encadenados, con dos espacios de nombres separados: el ordinario (variables, funciones, typedefs, constantes de enum) y el de *tags* (**struct**, **union**, **enum**). Al entrar a un bloque se apila un scope y al salir se desapila; la búsqueda recorre la cadena desde el scope actual hacia la raíz, y la detección de duplicados consulta solo el scope actual.

Las validaciones cubren, entre otras:

1. tipos nombrados existentes; prohibición de variables, parámetros y campos **void**; prohibición de funciones que retornen arreglos; cotas de arreglo enteras y mayores que cero; arreglos sin tamaño solo como parámetros;
2. distinción entre *lvalue* y *rvalue*, y rechazo de la asignación a *lvalues* de solo lectura (**const**);
3. compatibilidad de asignaciones e inicializadores, con conversiones aritméticas habituales, decaimiento de arreglo a puntero, conversiones desde y hacia **void** *, **null**, agregado de **const** y coincidencia exacta de firmas de puntero a función; compatibilidad nominal de agregados;
4. expresiones: aritméticas, comparaciones, lógicas, bit a bit, dirección (&), dereferencia (*), indexación, acceso por punto y por flecha, literales y llamadas;
5. llamadas a función: para las funciones declaradas en la unidad de traducción se verifica su carácter invocable, la aridad y la compatibilidad de los argumentos; las llamadas a nombres no declarados se asumen funciones externas y no se validan (ver la sección 4);
6. retornos: **void** no debe retornar valor, las funciones no-**void** deben retornar un valor compatible, y se exige que **todos** los caminos de una función no-**void** garanticen el retorno (cadenas **if/else** con todas las ramas cubiertas, **switch** con **default**, bucles siempre verdaderos sin **break**);
7. control de flujo: **break** solo dentro de un bucle o **switch**, **cnt** solo dentro de un bucle, y condiciones compatibles con su uso.

El inline C se trata como un bloque opaco: se copia a la salida pero no se verifica su tipo. Esta limitación es deliberada y se documenta como frontera del análisis.

Generación de código C

El generador (**CodeGenerator.c**) emite C determinista por la salida estándar con un contexto que sostiene el flujo de salida, el nivel de indentación (cuatro espacios por

nivel) y una bandera de error de E/S. La emisión se realiza en un orden estable de fases que respeta las reglas de declaración previa de C:

1. directivas de preprocesador;
2. inline C global;
3. enums;
4. agregados (`struct/union`);
5. typedefs;
6. prototipos de funciones;
7. variables globales;
8. definiciones de funciones;
9. `main`.

Este orden garantiza que los tipos se definan antes de usarse en los prototipos, que las funciones puedan llamarse entre sí mediante sus prototipos y que los inicializadores globales puedan referenciar funciones.

La traducción de tipos asigna cada `TypeKind` a su forma en C (`uint` → `unsigned int`, `uli` → `unsigned long int`, etc.). Los declaradores complejos —punteros, arreglos y punteros a función— se construyen de adentro hacia afuera (`_buildDeclarator`), siguiendo la sintaxis "inside-out" de C, agregando paréntesis donde la precedencia lo requiere (por ejemplo `(*nombre)(params)` para un puntero a función, o `(*nombre)[N]` para un puntero a arreglo). Otros detalles de traducción: los literales octales `0o...` se reescriben a la forma `0...` de C, `null` se emite como `((void *)0)`, los literales de arreglo se separan con comas y `main` se emite con la firma fija `int main(int argc, char *argv[])`.

La estrategia de paréntesis es deliberadamente conservadora: toda operación binaria y unaria (salvo el postfijo) se emite entre paréntesis, y el receptor de una indexación o de un acceso a miembro también se parentiza. El resultado es C válido y sin ambigüedad de precedencia, a costa de un poco más de texto.

3.3 Adicionales

Más allá del compilador, el proyecto incluye:

1. **Skill `writing-tel`**. El repositorio publica una *Agent Skill* que enseña a un agente (como Claude) a escribir TEL correcto. Se instala con `npx skills add https://github.com/BautistaPessagno/TEL --skill writing-tel`.

2. **Entorno reproducible.** Todo el desarrollo y las pruebas corren dentro de un servicio de Docker Compose, de modo que los artefactos de Linux generados coincidan con el entorno de corrección.
3. **Integración continua.** El repositorio expone un *badge* de pipeline que corre el build y la suite de tests sobre la rama de desarrollo.
4. **Detección de errores de memoria.** El build usa `-fsanitize=address`, de modo que las fugas y los errores de memoria hacen fallar los tests dentro del contenedor.
5. **Política de runtime.** TEL no provee un runtime propio. El C generado depende del estándar C, de los `#include` del usuario, del inline C y del *toolchain* C. Esta decisión mantiene el artefacto simple y portable; la contracara es que un programa que dependa de funciones externas debe proveer su implementación (o un inline C) para poder enlazarse en una prueba de integración ejecutable.
6. **Script de conveniencia `gtc.sh`.** El repositorio incluye `gtc.sh` (GNU tel compiler, nombre inspirado en `gcc`), que en un solo paso traduce un programa TEL a C con el compilador y compila el C resultante a un binario con `gcc`, todo dentro del contenedor. Resume el flujo de `src/main/bash/run.sh` seguido de `gcc` en un único comando: `bash gtc.sh programa.tel`.

Pruebas

Los tests son programas TEL ubicados en `src/test/c/accept/` (deben terminar con estado 0) y `src/test/c/reject/` (deben terminar con estado distinto de cero). La suite actual contiene **52 casos de aceptación** y **113 casos de rechazo**, ejecutados por `src/main/bash/test.sh`.

La cobertura abarca tres niveles. En el nivel **léxico/sintáctico**, los rechazos incluyen indentación inconsistente o de ancho inválido, *dedent* a un nivel inexistente, declaraciones al estilo C, `elif` huérfano, cabeceras malformadas de `for` y `switch`, inline C sin cerrar y comentarios de bloque sin cerrar. En el nivel **semántico**, los rechazos verifican redefinición de prototipos, identificadores desconocidos, llamadas con aridad o tipos de argumento incompatibles, asignaciones de tipo incompatible, falta de retorno garantizado, `switch` sin `default`, cadenas demasiado largas para un arreglo de `char`, retorno de un valor `void`, `break/cnt` fuera de contexto, indexación de un no-arreglo, dereferencia de un no-puntero, campo `void` en un agregado y punteros a objeto incompatibles. Entre los **casos de aceptación** se incluyen pruebas unitarias por construcción y casos de integración que combinan varias características, como un

programa de punteros y arreglos, un programa mixto y la resolución del problema *Two Sum*.

Programas de demostración

El directorio `demo/leetcode/` agrega ocho programas que resuelven problemas clásicos de estilo LeetCode: *Two Sum*, *Fizz Buzz*, *Fibonacci*, *Binary Search*, *Reverse Integer*, *Palindrome Number*, *Maximum Subarray* (algoritmo de Kadane) y *Climbing Stairs*. Cada programa se valida de extremo a extremo dentro del contenedor: se traduce de TEL a C con el compilador, el C resultante se compila con `gcc -Wall` y el binario se ejecuta, comparando su salida contra el resultado esperado del problema. Estos programas, además de servir como ejemplos de uso del lenguaje, son la base de la medición de tokens.

Además, el directorio `demo/parcial/` contiene la resolución en TEL del primer parcial de Programación Imperativa (PI) que nos tomaron en 2024, junto con el enunciado original (`primer_parcial_pi.pdf`). Son tres ejercicios, `ej1-firstN.tel`, `ej2-nqueens.tel` y `ej3-wordladder.tel`, que ejercitan arreglos, punteros de salida, etc. Sirven para mostrar que TEL es capaz de expresar problemas reales de estilo examen, más allá de los ejemplos de estilo LeetCode.

3.4 Dificultades Encontradas

Las principales dificultades durante el desarrollo fueron:

1. **Indentación significativa en Flex.** Flex no maneja bloques por indentación de forma nativa. Hubo que implementar una pila de niveles, generar `INDENT` y `DEDENT` de forma explícita y vaciar los `DEDENT` pendientes al fin de archivo para que Bison supiera dónde empieza y termina cada bloque.
2. **Ambigüedad del operador `->`.** La flecha aparece tanto en el operador de acceso por puntero como en el tipo de retorno de las funciones. Se resolvió con la *start condition* `AFTER_POSTFIX`, que solo interpreta `->` como acceso cuando viene después de un identificador o de un cierre.
3. **Sintaxis de declaradores en C.** Los punteros, los arreglos y, sobre todo, los punteros a función tienen una sintaxis delicada. Generar declaradores válidos exigió construirlos de adentro hacia afuera y parentizar con cuidado.

4. **Decaimiento de arreglos a punteros.** Hubo que separar la compatibilidad de tipos de TEL del decaimiento de arreglos a punteros que ocurre cuando se pasan como parámetros.
5. **Garantía de retorno por todos los caminos.** Determinar si una función no-`void` retorna en todos sus caminos requirió analizar cadenas `if/else`, `switch` con `default` y bucles siempre verdaderos.

4. Limitaciones

El compilador es correcto dentro del subconjunto y del alcance que valida, pero hay fronteras conocidas que conviene explicitar. Que un programa TEL pase el análisis léxico, sintáctico y semántico no garantiza que el C generado compile: TEL valida la coherencia interna de la unidad de traducción, no su correctitud frente al toolchain de C.

Las principales limitaciones son:

1. **No se validan los prototipos de funciones externas ni de librería.** El análisis semántico solo conoce las funciones declaradas en la propia unidad de traducción. Cuando una llamada apunta a un nombre no declarado, el compilador la asume una función externa y únicamente valida las expresiones de sus argumentos, sin verificar que la función exista, ni su aridad, ni los tipos de sus parámetros, ni su tipo de retorno. En consecuencia, si una persona escribe una función inexistente, o invoca mal una función de librería, TEL no emite ningún error y genera C de todas formas. El error recién aparece al compilar ese C con `gcc` (declaración implícita, conflicto de tipos o error de enlazado). TEL no lee los headers de los `#include`, de modo que no puede conocer las firmas que estos aportan.
2. **El inline C es un bloque opaco.** El código C embebido entre comillas invertidas se copia a la salida sin verificación de ningún tipo. Cualquier error dentro de un bloque inline C se traslada al C generado y solo lo detecta `gcc`.
3. **Literales negativos en literales de arreglo separados por espacios.** Esta limitación se descubrió al validar los programas de demostración (sección 3.3). Como los elementos de un literal de arreglo se separan con espacios, un literal con valores negativos como `{-2 1 -3 4}` se interpreta como una resta entre elementos consecutivos (`1 - 3`), produciendo un arreglo con menos elementos de los esperados. Es una limitación del generador de código que se sortea inicializando el arreglo elemento por elemento.

TEL delega en el toolchain de C la verificación de todo lo que queda fuera de la unidad de traducción.

5. Futuras Extensiones

A partir del estado actual, las extensiones naturales son:

1. **Dirección C $\rightarrow$ TEL.** Completar la transformación biyectiva implementando el sentido inverso, de modo de poder comprimir código C existente a TEL.
2. **Inferencia de tipos.** Permitir declaraciones como `x = 5` sin tipo explícito, lo que reduciría aún más los tokens.
3. **Ampliar el subconjunto de C.** Incorporar más construcciones a medida que se demuestren valiosas para los flujos de trabajo agénticos.

6. Conclusiones

El Stage III cumple su objetivo: el compilador recorre el pipeline completo desde el texto TEL hasta el código C, con análisis léxico, sintáctico, semántico y generación de código funcionando de extremo a extremo. El generador produce C determinista y compilable para el subconjunto documentado, y el análisis semántico rechaza los programas que el parser por sí solo no puede distinguir.

Las mediciones muestran que TEL reduce de forma consistente la cantidad de tokens respecto del C equivalente, con el mayor ahorro en los tipos compuestos y en las directivas, lo que valida la premisa del dominio. La suite de 52 casos de aceptación y 113 de rechazo, junto con casos de integración como *Two Sum*, respalda la robustez del compilador y deja en evidencia el comportamiento del lenguaje frente a programas reales.

7. Referencias

1. Cátedra ATLC. [\[icon\] \(2025-09-03, v1.0.0\) Análisis Léxico](#) .
2. Cátedra ATLC. [\[icon\] \(2024-05-08, v0.1.0\) Análisis Sintáctico](#)
3. Cátedra ATLC. [\[icon\] \(2024-09-25, v0.2.0\) Análisis Semántico](#)
4. Cátedra ATLC. [\[icon\] \(2024-05-16, v0.1.0\) Generación de Código](#)

5. Cátedra ATLC.  (2024-05-16, v0.1.0) Runtime
6. OpenAI. Tokenizer. <https://platform.openai.com/tokenizer>.
7. Repositorio del proyecto TEL. <https://github.com/BautistaPessagno/TEL>.

8. Bibliografía

1. J. E. Hopcroft, R. Motwani y J.D. Ullman. Introduction to Automata Theory, Languages and Computation (3ª ed.). USA. Addison—Wesley. 2006.
2. *Flex – The Fast Lexical Analyzer Generator*, documentación oficial del proyecto GNU Flex.
3. *Bison – GNU Parser Generator*, manual oficial del proyecto GNU Bison.